BloodHound

B) BloodHound

This section provides the technical documentation for **Module 2: AD Post Enumeration**, specifically focusing on the use of **BloodHound** to visualize attack paths and identify high-value targets within the domain.

Module 2: AD Post Enumeration

Methodology: Attack Path Visualization with BloodHound

1. Overview

BloodHound uses graph theory to reveal the hidden and often unintended relationships within an Active Directory environment. By ingesting data collected from SharpHound or similar collectors, it allows an attacker to identify complex attack paths from a standard user to Domain Admin.

2. Lab Execution & Analysis

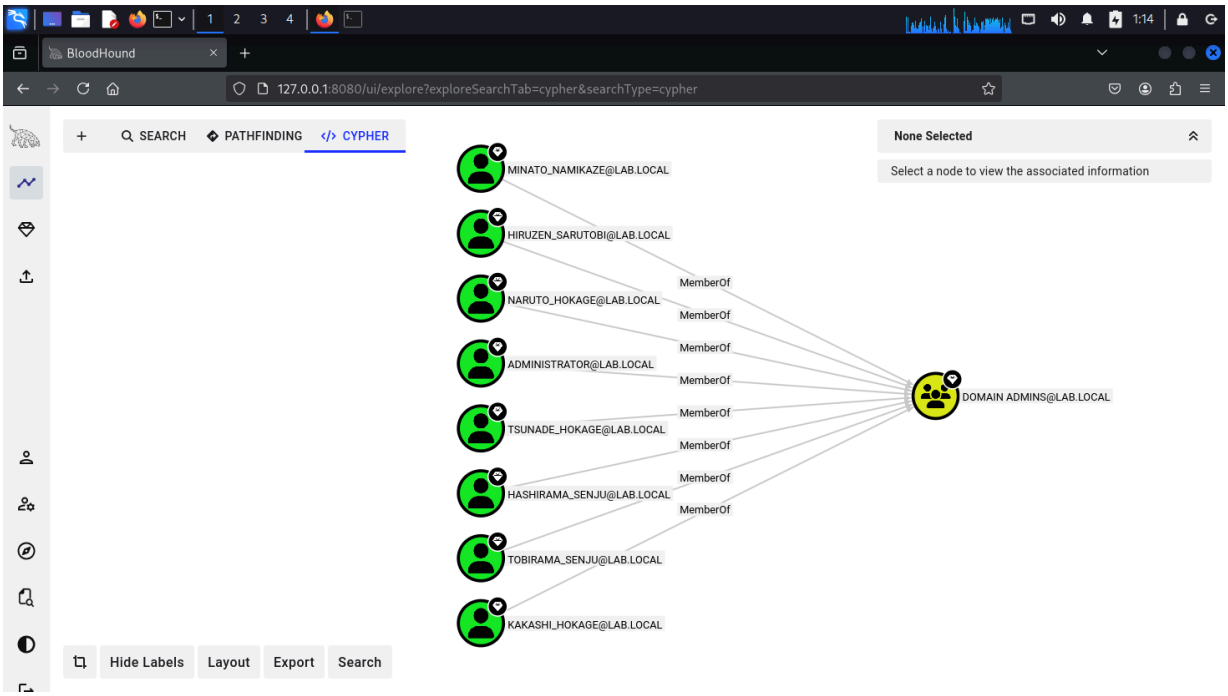
Step 1: Environment Initialization
Before launching the BloodHound GUI, the **Neo4j** graph database must be started to handle the data backend.

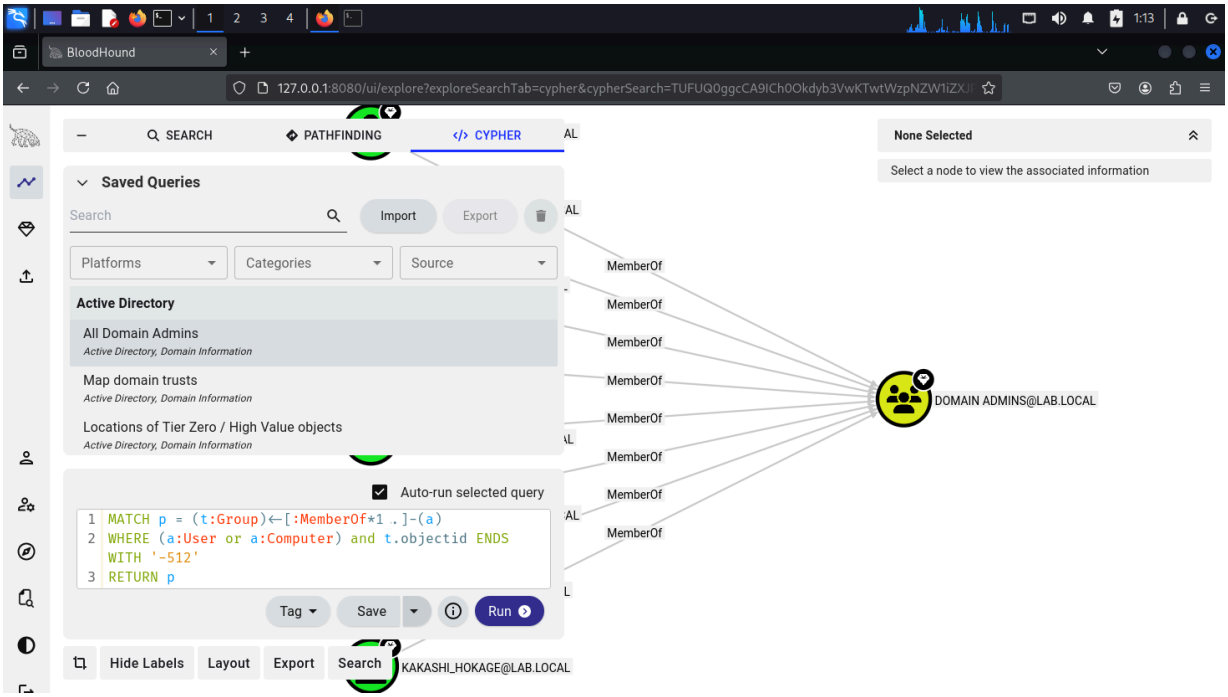
- The terminal shows the Neo4j service starting up at PID 41393, followed by the initialization of the BloodHound application.

Step 2: Mapping High-Value Group Memberships
We begin by visualizing the members of the most privileged groups in the domain to identify our ultimate targets.

- **Findings:** Identified several high-value accounts belonging to the **DOMAIN ADMINS@LAB.LOCAL** group, including `MINATO_NAMIKAZE`, `HIRUZEN_SARUTOBI`, `NARUTO_HOKAGE`, and `HASHIRAMA_SENJU`.

- The graph shows the `MemberOf` relationships connecting these specific user objects to the Domain Admins group object.

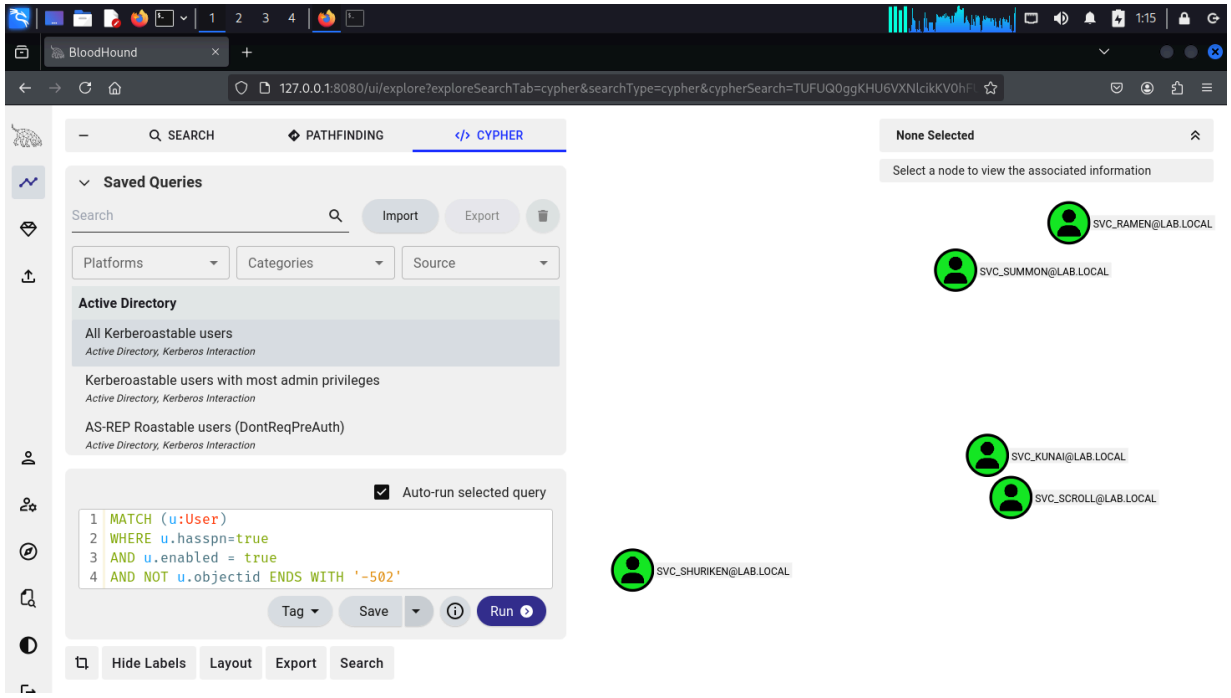




Step 3: Identifying Kerberoastable Service Accounts

Using built-in Cypher queries, we filtered for accounts with Service Principal Names (SPNs) that are not administrative but may hold useful permissions.

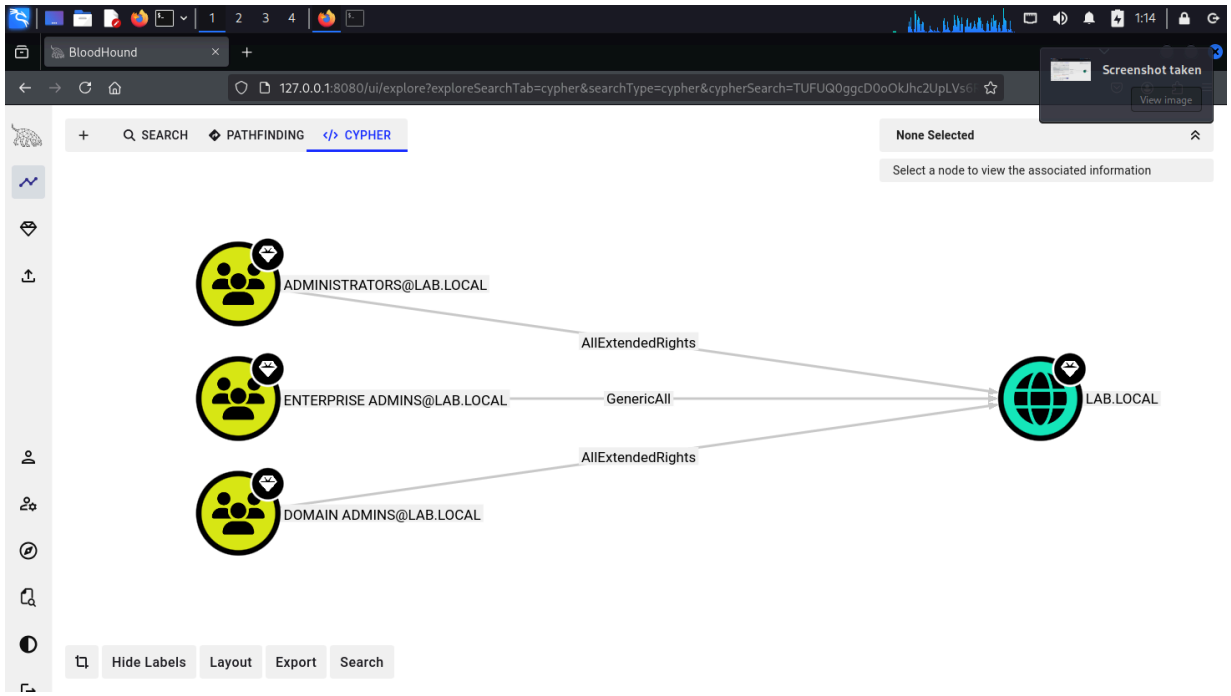
- **Findings:** The query identified multiple service accounts: `SVC_RAMEN`, `SVC_SUMMON`, `SVC_KUNAI`, `SVC_SCROLL`, and `SVC_SHURIKEN`.
- **Impact:** These accounts are primary targets for the **Kerberoasting** attack (Module 4) to crack their passwords offline.
- The UI displays the results of the "All Kerberoastable users" query.



Step 4: Analyzing Dangerous Domain Rights

We analyzed the global permissions assigned to administrative groups over the domain object itself.

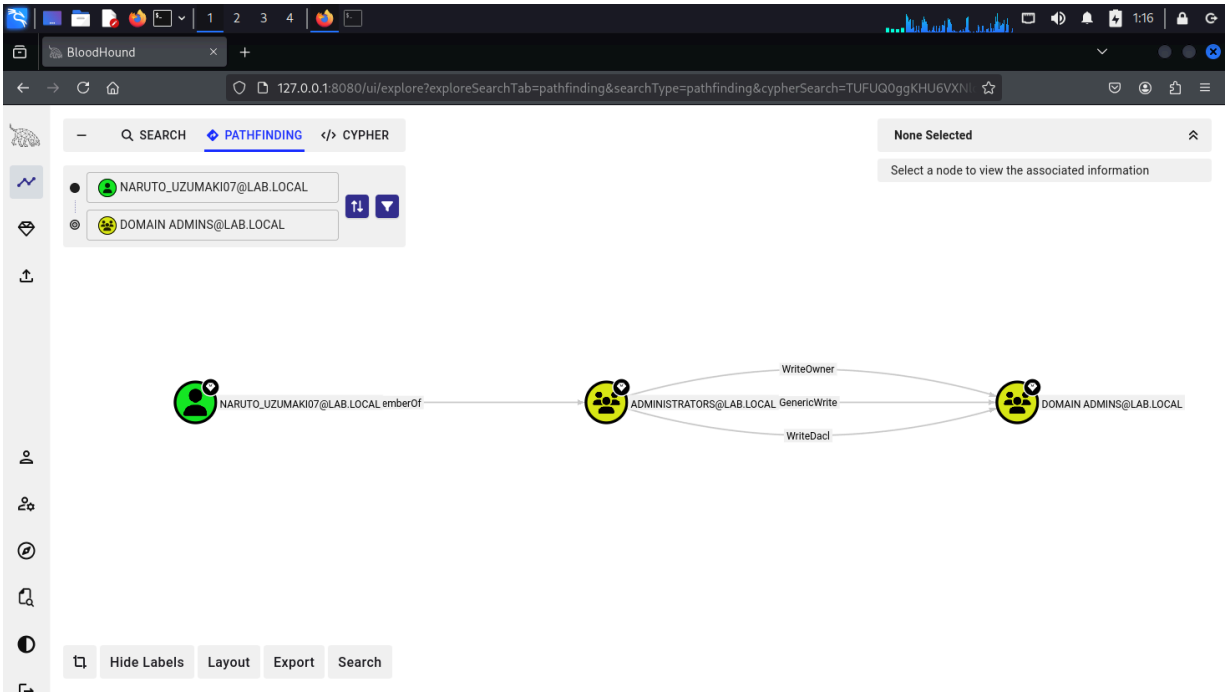
- **Findings:** The **ADMINISTRATORS**, **ENTERPRISE ADMINS**, and **DOMAIN ADMINS** groups all hold dangerous rights such as `AllExtendedRights` and `GenericAll` over the `LAB.LOCAL` domain.
- These edges represent the ability to perform highly sensitive tasks, such as DCSync attacks or modifying domain-wide ACLs.



Step 5: Pathfinding to Domain Admin

The core of this enumeration phase involved finding a direct attack path from our compromised user to the Domain Admin group.

- **Path Discovered:** `NARUTO_UZUMAKI07` → `MemberOf` → `ADMINISTRATORS` → `GenericWrite/WriteDacl/WriteOwner` → `DOMAIN ADMINS`.
- **Critical Finding:** Our current user, `NARUTO_UZUMAKI07`, is a member of the **ADMINISTRATORS@LAB.LOCAL** group. This group has `GenericWrite`, `WriteDacl`, and `WriteOwner` permissions over the **DOMAIN ADMINS@LAB.LOCAL** group.
- **Impact:** This is a "one-hop" path to full Domain Admin. We can use these rights to add ourselves directly to the Domain Admins group or modify its members' attributes.
- The pathfinding graph clearly illustrates this critical privilege escalation vulnerability.



BloodHound ingests data (collected via SharpHound or similar tools) into a **Neo4j graph database** to visualize abusable relationships between users, groups, computers, and the domain. This allows us to map out "lateral movement" steps from a standard user to high-value targets.

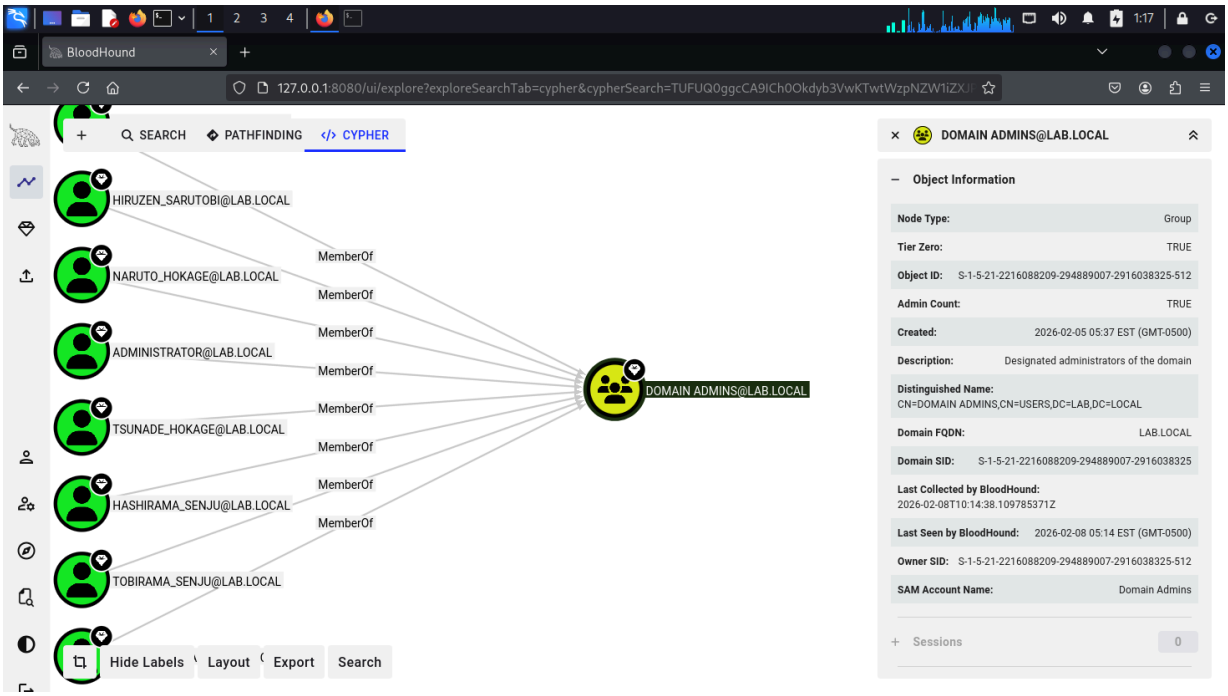
2. Lab Execution & Analysis

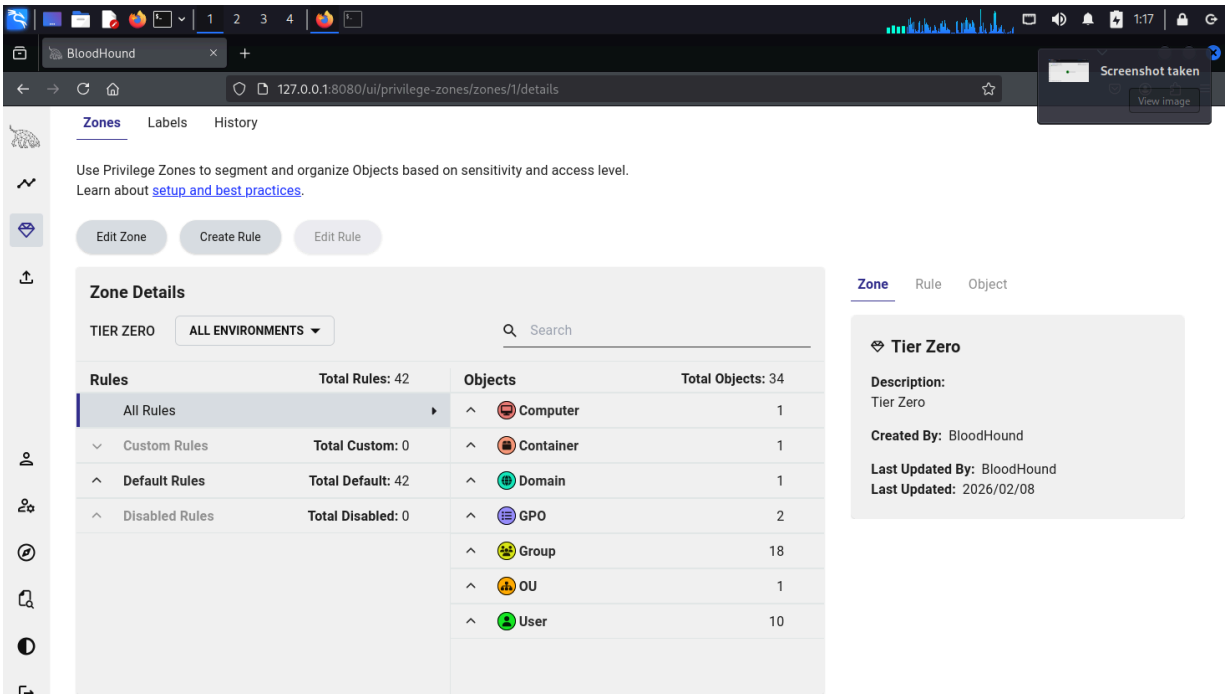
Step 1: Database Initialization The analysis environment is prepared by starting the Neo4j service and the BloodHound application.

- **Source (1.png):** Shows the **Neo4j** console starting (PID 41393) and BloodHound preparing to launch.
- **Status:** Database online; ready for data ingestion.

Step 2: Identifying "Tier Zero" Objects We first identify the environment's most critical assets —"Tier Zero" objects—which have full control over the domain.

- The "Tier Zero" dashboard summarizes the total count of critical objects (1 Domain, 18 Groups, 10 Users).
- **High-Value Target:** `DOMAIN ADMINS@LAB.LOCAL` is identified as a Tier Zero group.
- **Node Info (11.png):** Detailed inspection of the `ADMINISTRATOR@LAB.LOCAL` user confirms it is a **Tier Zero** principal with `Admin Count: TRUE`.





Step 3: Visualizing High-Value Group Membership Using built-in queries, we map the direct and nested memberships of administrative groups.

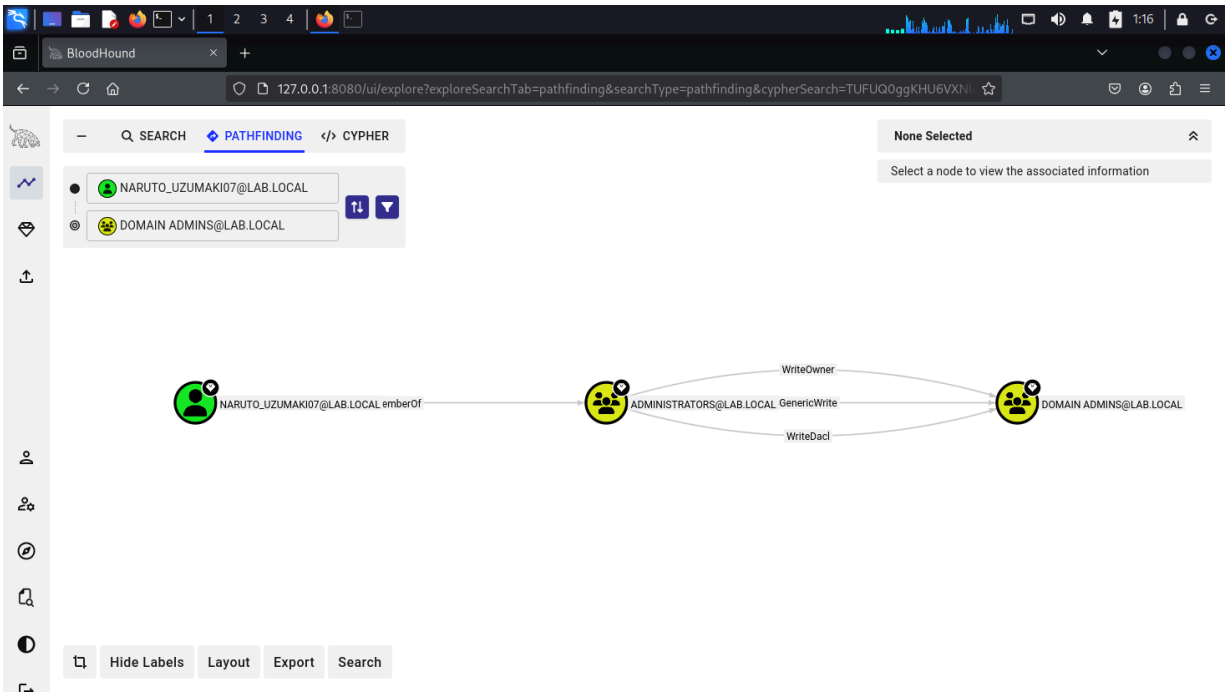
- **Findings:** Identified members of the **Domain Admins** group, including `MINATO_NAMIKAZE`, `NARUTO_HOKAGE`, and `HASHIRAMA_SENJU`.

- **Source (2.png, 3.png):** Graph view showing the `MemberOf` relationship connecting these users to the administrative group.

Step 4: Kerberoasting Attack Surface We executed a custom Cypher query to identify all users with a **Service Principal Name (SPN)** who are also enabled accounts.

- **Query:** `MATCH (u:User) WHERE u.hasspn=true AND u.enabled=true RETURN u`.
- Discovered several service accounts: `SVC_RAMEN`, `SVC_SUMMON`, `SVC_KUNAI`, `SVC_SCROLL`, and `SVC_SHURIKEN`.
- **Impact:** These accounts are vulnerable to offline password cracking via Kerberoasting (Module 4).

Step 5: Critical Attack Path Discovery The core finding of this phase is a "Shortest Path" to Domain Admin from our currently compromised principal.

- **Path:** `NARUTO_UZUMAKI07` → `MemberOf` → `ADMINISTRATORS` → `WriteDacl/WriteOwner/GenericWrite` → `DOMAIN ADMINS`.
- **Vulnerability:** Our user is in the local `ADMINISTRATORS` group, which has dangerous **DACL permissions** over the `DOMAIN ADMINS` group.

This path reveals we can directly modify the `DOMAIN ADMINS` group to add ourselves or take ownership of it.